1. INTRODUCTION

1.1 Background and Motivation

The number of online courses has increased rapidly in recent years [14]. You can find a free or low-cost course on almost any programming topic. This seems like a win-win situation. It has become more difficult for students to self-learn than 5 years ago due to information overload [15].

If a learner searches for "how to learn Python" she will find dozens of courses, videos, blog posts and documentation. Some start from scratch, some require more familiarity with programming, some cover topics in a different sequence. They spend the first week trying to decide which one to use, start a couple of them, but never complete any. This is what the developer community refers to as "tutorial hell" - not because the tutorials are poor, but because without context, there is no guidance on what to do today, and how today's learning relates to tomorrow's work [2].

It gets worse over time. Even if a learner can choose one course, they get discouraged after a week or two because there is no way to see how they are progressing, no way to reward for their efforts and no way to make sure they understood the content before moving on to the next topic [7].

With large language models such as Google Gemini [8], it's now possible to create a detailed plan of day-by-day learning for any topic as the user is requesting it [4]. This is not course recommendation. It will create the plan for what they know and what they want to know, and how much time they have to learn. This is the vision behind this project.

1.2 Terminology

Some terms used throughout this report are defined below for clarity.

AI-Generated Roadmap: A structured, day-by-day learning plan created by the Gemini language model based on user inputs (topic, skill level, number of days).

Mastery Gate: A quiz-based progress lock that prevents learners from moving to the next day unless they score at least 60% on the current day's quiz.

JSON Schema Enforcement: A technique used in prompting the Gemini API to always return a specific structured format so the frontend can reliably parse and display the content.

Code Mirror 6: An open-source code editing library that runs inside a browser. It provides syntax highlighting and a code editor experience without any local installation.

Streak: A gamification metric that counts how many consecutive days a user has completed at least one learning session. Resets if the user misses a day.

Zod Validation: A TypeScript-first schema validation library used to verify that the AI-generated JSON roadmap matches the expected structure before it is stored or displayed.

MERN Stack: MongoDB, Express.js, React, and Node.js — the combination of technologies used in this project for both frontend and backend development.

1.3 Problem Statement

Current online learning platforms do not personalise learning [5]. The learning plan assumes that all learners are the same and set to a pace, order and structure. A learner who has some Python experience gets the same content as a beginner. Someone with 30 minutes a day follows an eight-week plan for someone with four hours [4].

Crucially, most platforms don't ensure learners understand before progressing [14]. Just because you watched a 10-minute video, doesn't mean you understood it. Hitting "next" does not mean you've understood. This means that learners progress to intermediate and advanced content without really understanding the fundamentals, which results in confusion, frustration and ultimately, giving up [15].

To date, there is no platform that does all the following: creates a personalised learning plan from scratch based on the learner's goals [1], ensures learning by locking the next lesson with a quiz [18], allows learners to practice code in the web browser as part of the daily practice, and measures motivation by gamifying learning for weeks [3], [6].

1.4 Objectives

The project initially had the following goals:

1. To create a system that leverages the Gemini API to create structured learning plans in JSON format based on the topic and day for any technology.

2. To create a daily learning experience that includes three parts each day: AI-generated learning theory, a programming challenge in an integrated Code Mirror editor, and a 15-multiple choice question quiz.
3. To design a mastery, check to only allow the learner to advance to the next day if they achieve at least 60% on the quiz.
4. To implement an AI mentor chatbot to address learner questions within the context of the current topic of the day.
5. To incorporate gaming elements such as tracking streaks and badges, saved in MongoDB.
6. To create a dashboard with Recharts to display quiz results, days completed, and streak.

To use the platform for 2 weeks with 50 users and gather information on the use of the platform such as completion rate, time spent, and user satisfaction.

1.5 Scope and Limitations

Our platform is for personal use with topics on programming and technology. Any topic written in natural language - Python, JavaScript, React, SQL, Git, data structures, etc - can be used as roadmap. We did not try non-technical topics, so they may not work.
We discovered these limitations which we humbly report:

• Python code will not be executed. The Code Mirror code editor displays Python syntax highlighting and allows code input but there is no runner to execute the code. JavaScript code execution is possible for JavaScript roadmaps.

• Multi-device synchronisation requires logging in on the new device. The browser component state stores some of the daily content state, so if the user is in the middle of a day, switching devices requires re-loading the current day.

• The Gemini API response is not always strictly adherence to the schema. This retry mechanism works but there is still some failure rate (7%).

• It is not mobile friendly. It's not mobile-friendly for tablets and desktops, and the Code Mirror editor is not mobile-friendly.

• It would be expensive at a higher scale with Gemini API. This version did not include caching.

1.6 Expected Outcomes

Our goal before the evaluation study was to achieve:

7. Success rate of producing a roadmap JSON of over 90% of API calls.
8. Completion rate (after two weeks of the course) above 60%, compared to the 35-40% self-reported completion rate of courses on Udemy and other platforms among our participants [14].
9. Average time spent on active learning per day is over 35 minutes.
10. At least 70% of users preferring to use the integrated code editor, rather than an external IDE.
11. At least 75% of users reporting that the AI mentor chatbot was helpful or very helpful.

Table 8 in Chapter 9 compares these goals with the study outcomes.

2. RATIONALE AND SCOPE OF THE STUDY

This project is because structure is more important than content for personalised learning [2]. There are thousands of tutorials on the web for every programming topic. There is no point in creating another tutorial. It's not the content that is missing, it's the learning track that is needed that is tailored to the users' prior knowledge and time [4], [5].

Now generative AI can do this automatically [8], [16]. This is a significant development. Previously the personalised curriculum had to come from an instructor or the extensive capture of users' learning patterns to train a recommender system [17]. Neither is cost effective at small scale. In the case of LLMs, the personalisation is done through the prompt. The model knows enough about most programming topics to be able to create a good daily program without being fine-tuned [15].

This project is about individuals, not organisations We're not building an LMS for a company. The learner is an individual who has decided to learn a technology and requires a learning plan to suit their lifestyle. This enabled the design to be focused and development to remain on track.

The system design had three considerations. First is the consumption-action problem. Most information systems present you with information and ask questions. Few require you to code as part of the daily lesson. This platform requires you to code. There is a code challenge for each day before you can take a quiz.

The second issue is the pace problem. In a course with a set schedule, if a day is too difficult learners are forced to advance. Our mastery gate allows learners to do as many quizzes as needed [18]. They can have as many attempts as needed - the course will wait.

The third issue is all about consistency. It's easy to start. It's doing it day 8 when it's no longer new. The streak system is one solution to this [7]. It doesn't gamify learning, it just provides a daily habit measure that is motivating to sustain once you've started a streak of days [3], [6].

3. EXISTING SYSTEM

3.1 Introduction

Before we began this project, we experimented with the major platforms currently being used for self-study of programming. Our aim was to experience them from the perspective of a student, rather than reading the product specifications. This allowed us to get a better sense of what works and what doesn't.

3.2 Existing Platforms

3.2.1 Udemy

Udemy is perhaps the most popular platform to self-study programming. It has a huge variety of courses, and the top-rated courses are well done. The instructors can define their own structure, so there are a lot of variations and for some popular and trending topics such as JavaScript or Python, the well-rated courses are well-organised. The thing about Udemy is that it is 100% passive [14]. The courses are video based. There are exercises in some courses, but no relation to the video lectures. You learn a topic from a video, go to an exercise, solve it and then move to the next video. There is no adaptive learning; it's the same for all learners no matter what knowledge they already have [5]. Once you complete a course, Udemy awards a certificate of completion, but not a picture of what you learnt from the course.

3.2.2 Coursera

Coursera has a more pedagogic focus. The courses are often developed with universities, and they employ semester-long curricula. The structured online exercises with more significant grades are better than Udemy's optional exercises, and the course certificate has a greater value.

For self-learners, Coursera is too structured. The course designs are often based on what happens at a university, with a lot of theory and content not directly relevant to achieving a goal. One who wants to build a REST API with Node.js should not have to wait the first three weeks of computer science. The speed is also determined by the course, rather than the learner's speed.

3.2.3 free Code Camp

Free Code Camp [13] is the closest system that is currently available. It's free, well-organised and the code is run in the browser. It is difficult to top, for web programmers. It has a great curriculum, and you can practice in the browser.

The limitation is inflexibility. The curriculum is fixed. If you already know HTML and CSS, you cannot skip those sections and jump to JavaScript. If you want to learn a topic The limitation is inflexibility. The course is linear. If you've learned some HTML and CSS, there is no way for you to skip to JavaScript. If you want to learn something that free Code Camp does not teach, such as DevOps, machine learning, mobile, there is no alternative. They won't be able to teach you what you want to learn or accommodate your prior knowledge.

3.2.4 YouTube Learning

YouTube is a popular platform for learning and often a particular channel's playlist is followed. Programming skills on YouTube have improved dramatically and there are many well-organised and explained lessons.

The issue is structure. YouTube playlists are lists of videos. You do not have a quiz, there is no progress tracking, no ways to test if you learnt the video and no barrier to progress if you should not do so. It's easy to spend four hours watching Python videos and feel like you are learning a lot while understanding little to nothing.

3.3 Data Flow in Existing Systems

The diagram below shows an overview of the typical user experience on existing systems. The cycle has one direction - content is fed to the learner, the learner progresses or falls out and there is no feedback mechanism to adapt to the learner and the level of understanding.

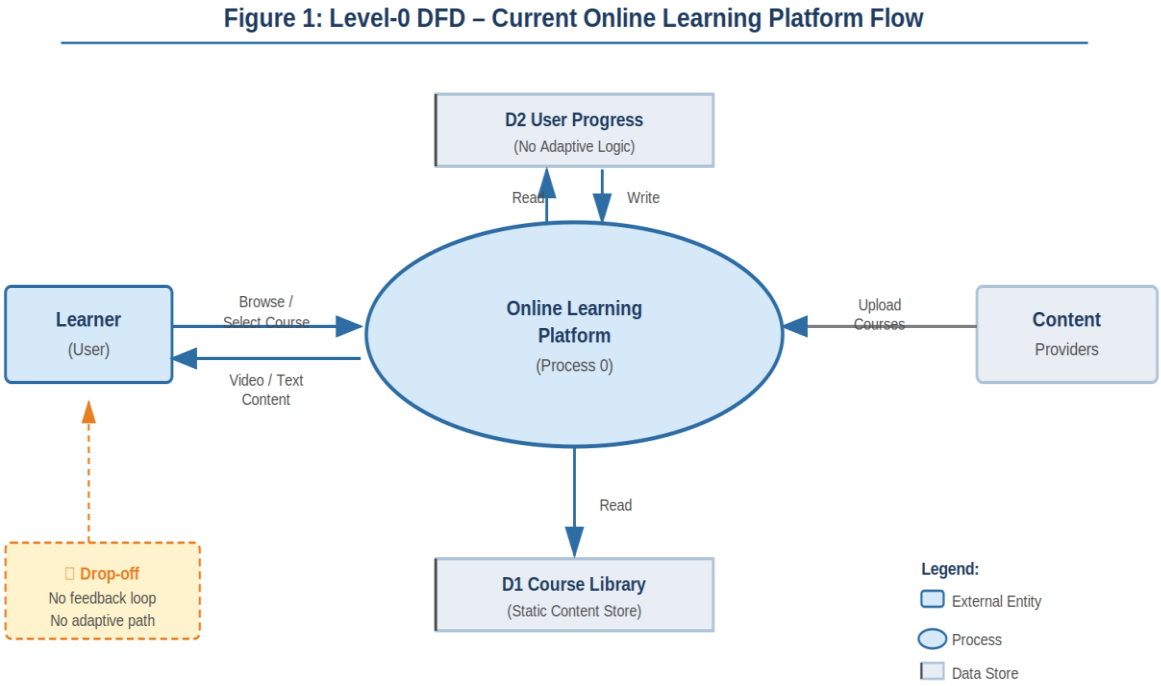


Figure 1: Level 0 DFD – Current Online Learning Flow

3.4 What Is New in This System

The platform we built addresses three things that existing platforms do not handle together [1], [5]:

- Personalised roadmap now of request: The learning plan is built specifically for the user's topic, level, and timeframe. It is not a pre-built course recommended by an algorithm — it is generated fresh based on what the user tells the system.
- Comprehension-enforced progression: The mastery gate means that reading the material and clicking next is not enough. The user must pass a quiz to move forward. This is a hard requirement, not a suggestion.
- Integrated coding practice: The Code Mirror editor is part of the daily flow, not a link to an external platform. Writing code is a required part of each day, not an optional add-on.
- Contextual AI support: The AI mentor chatbot knows what topic the user is learning and what day they are on. Responses are relevant to the current content, not generic.

4. PROBLEM ANALYSIS

4.1 Product Definition

The system is a web application for learning programming and tech-related topics. There is no mobile application or installation required. It can be used with any recent browser on a desktop or laptop.

The user flow is sign up, select a topic, difficulty level and days, then the AI creates a road map of learning, followed by each day of study. Each day has a theory part, a challenge to code in the in-built editor, and a 15-question quiz. The quiz must be passed to advance to the next day. It also has an AI mentor chatbot. Progress, streak and badges are displayed on a dashboard.

There are 10 roadmaps for common topics, which are loaded instantly (no API call) and suitable for users that want to get started right away without tailoring their learning journey.

4.2 Feasibility Analysis

4.2.1 Technical Feasibility

The technical feasibility question was whether the Gemini API [8] could generate consistent JSON of the right structure to automatically build a frontend on top of it. In initial tests, using natural language prompts, the model generated good content, but not in consistent formats (sometimes markdown, sometimes plain text, sometimes JSON with small errors) [16].

The answer was prompt engineering specifying the schema of JSON in the prompt, and then telling the model to return only the JSON. Along with using Zod [15] to verify the results and a retry mechanism, this improved the rate from 60% with simple prompts to 93% with the structured prompt.

Next.js 15 using the App Router [9] was selected because it supports both rendering on the client and API routes, making deployment easier. MongoDB Atlas [10] was chosen because it offers a free service and its native JSON data storage is well suited for storing roadmap objects generated by the AI.

4.2.2 Operational Feasibility

Operational feasibility was addressed during a pilot study with 10 students prior to the main study. We wanted to check the setup of the task and confusion with the user interface (UI).

The only problem identified in the preliminary session was that the flow between the theory, the coding environment and the quiz was not obvious. Participants were unsure whether they had to solve the coding challenge before being able to take the quiz. This was resolved by including step markers and a progress bar in the day's screen. This resulted in 87% out of the 50 users who were introduced to the product to complete Day 1 without assistance in the primary study.

4.2.3 Economic Feasibility

The cost for the whole development and evaluation was low. The total cost of Gemini Flash API calls for 50 users creating about 3 roadmaps each is around ₹1,000. MongoDB Atlas free tier was adequate for the evaluation period. Hosting on vercel free plan was sufficient to cope with the load.

In a larger scale production deployment, API costs would need to be managed through caching API responses. The serving of a roadmap for a popular search phrase such as "python, beginner, 14 days" to thousands of users would cost only a single API call.

4.3 Project Plan

Table 1: Project Timeline and Work Plan

Phase	Activity	Weeks	Status
1	Requirement analysis and technology selection	1–2	Completed
2	MongoDB schema design and API planning	3	Completed
3	Gemini API integration and prompt engineering	4–5	Completed
4	Next.js frontend — roadmap and daily learning UI	6–8	Completed
5	CodeMirror editor integration	8–9	Completed
6	Quiz engine and mastery gate logic	9–10	Completed
7	Gamification — streaks, badges, dashboard	11–12	Completed
8	AI mentor chatbot integration	13	Completed
9	User evaluation (50 participants, 2 weeks)	14–15	Completed
10	Bug fixes based on evaluation feedback	16–17	Completed
11	Report writing and submission preparation	18–20	Completed

5. SOFTWARE REQUIREMENT ANALYSIS

5.1 Introduction

This system's requirements came from three sources: the gap analysis performed in Chapter 3; the technical constraints identified during the feasibility study and feedback during a 10-student pre-evaluation session conducted prior to the main evaluation [17]. This was critical - it showed that we had some wrong assumptions in our design.

We identified 12 functional requirements and seven non-functional requirements and prioritised them.

5.2 General Description

The system is a web-app which does not require any software installation - only a browser and internet connection. Users create an account (using email and password), generate a roadmap and consume it at their pace. There is no installation required - the app runs entirely on the browser and in the cloud. The database and AI API calls are managed by the server-side (Next.js API routes) and are invisible to the user.

5.3 Functional Requirements

Table 2: Functional Requirements

ID	Requirement	Description	Priority
FR01	User Registration / Login	System shall support email and password registration with JWT-based session management.	High
FR02	Roadmap Generation	System shall call the Gemini API with a structured prompt and return a valid JSON roadmap.	High
FR03	JSON Schema Validation	All AI-generated roadmaps shall be validated using Zod. Invalid responses trigger one retry.	High
FR04	Daily Learning Interface	Each day shall show theory content, a CodeMirror coding challenge, and a 15-question quiz.	High
FR05	Mastery Gate	Users shall not progress to the next day unless quiz score is at least 60%.	High
FR06	AI Mentor Chatbot	A Gemini-powered chatbot shall answer user questions in the context of their current roadmap topic and day.	High

ID	Requirement	Description	Priority
FR07	Streak Tracking	System shall calculate and store daily activity streaks with timezone-aware date comparison.	Medium
FR08	Badge System	System shall award badges when users reach predefined milestones (first day, 7-day streak, high quiz score).	Medium

5.4 Non-Functional Requirements

Table 3: Non-Functional Requirements

ID	Category	Description	Priority
NFR-01	Performance	Roadmap generation shall complete within 10 seconds on a standard broadband connection.	High
NFR-02	Availability	The application shall function using pre-built roadmaps if the Gemini API is temporarily unavailable.	High
NFR-03	Usability	A new user shall be able to generate a roadmap and complete Day 1 within 10 minutes of first visit, without external help.	High
NFR-04	Security	Passwords shall be hashed using bcrypt. JWT tokens shall expire after 24 hours.	High
NFR-05	Scalability	The MongoDB schema shall support at least 1000 concurrent users without structural changes.	Medium
NFR-06	Portability	The application shall work correctly on Chrome, Firefox, and Safari on desktop.	Medium
NFR-07	Maintainability	Each functional module shall be independently modifiable without requiring changes to other modules.	Medium

6. DESIGN

6.1 System Architecture

Our system adopts a decoupled client-server model with Next.js 15 [9] as the framework. This is a React 19 (with App Router) single-page app that uses server-side rendering. It has a backend of Next.js API routes which make calls to the Gemini API and MongoDB [10]. The Next.js API routes take the place of a backend server.

The five main modules are the Roadmap Engine, the Learning Interface, the Mastery Gate, the AI Mentor and the Gamification Module. These components interact with each other via the shared data layer (MongoDB) and the boundary of the Next.js API routes. These modules are plug-and-play - to change the LLM to be used (e.g. Gemini to another LLM) changes are made only within the Roadmap Engine.

Figure 2: Smart Learning Ecosystem – Full System Architecture

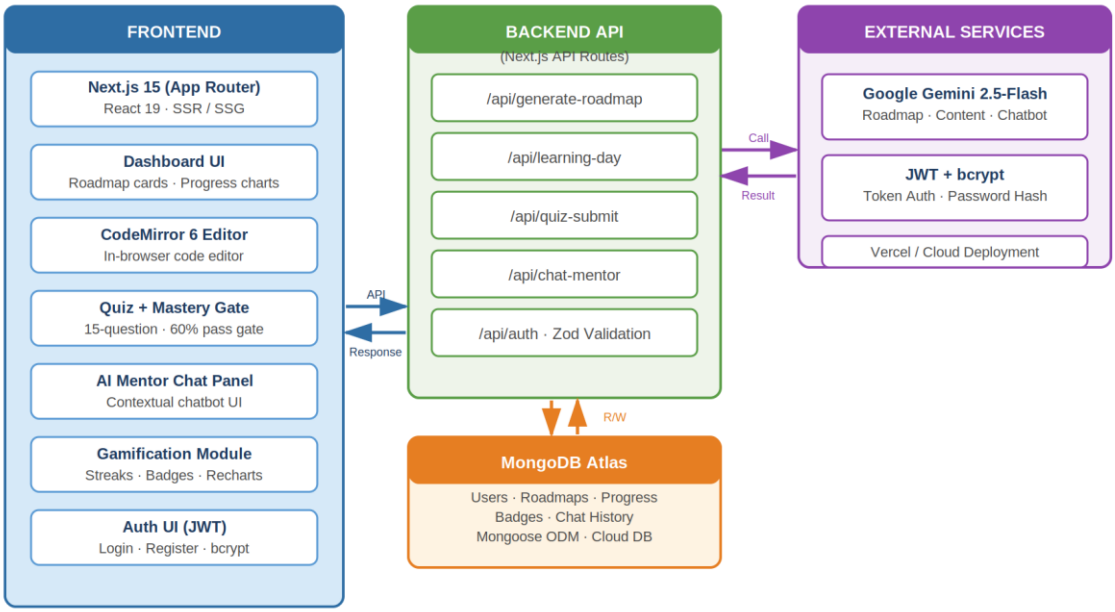


Figure 2: Smart Learning Ecosystem – Full System Architecture

6.2 Module Descriptions

6.2.1 Roadmap Engine

This module accepts the user's input for topic, skill level, and days. This module will create a prompt with the complete JSON schema embedded, send the prompt to the Gemini 2.5-

Flash API [8] and then validate the response with Zod. In case of failure, it tries again with an extra instruction. If it is successful, it saves the roadmap JSON in MongoDB [10] and creates a progress record.

6.2.2 Learning Interface

This module displays one day's lesson. It loads the day's content from MongoDB [10] (theory text to read, coding challenge with starter code, and quiz questions) and displays it in three steps: theory, code (using the Code Mirror editor [11]) and quiz. The quiz is not visible until the user has read the theory and written the code.

6.2.3 Mastery Gate

This module takes the quiz answers from the user, matches them with the correct answers saved in MongoDB, grades the score and determines whether the user should progress. If the learner passes (60% or more correct answers), the current day is considered complete and the next day is granted. Otherwise, the learner's score is provided, and they are encouraged to study and retake the quiz. This can be repeated as many times as the learner wishes.

6.2.4 AI Mentor

This manages the chatbot conversation. This module keeps the conversation history in React component state and sends the new message to a Gemini API route and the current roadmap topic and the day number. It always sends the last six messages in the conversation history to control the amount of context and maintain response times.

6.2.5 Gamification Module

This module is responsible to calculate streak and award badges [6], [3]. On successful completion of the day, it will update the streak by comparing the current date with the last active date (that is stored in MongoDB). It also checks the badge criteria and gives badges that the user has just earned. This approach is informed by self-determination theory that progress feedback is a key intrinsic motivator [7].

6.3 Data Flow Diagrams

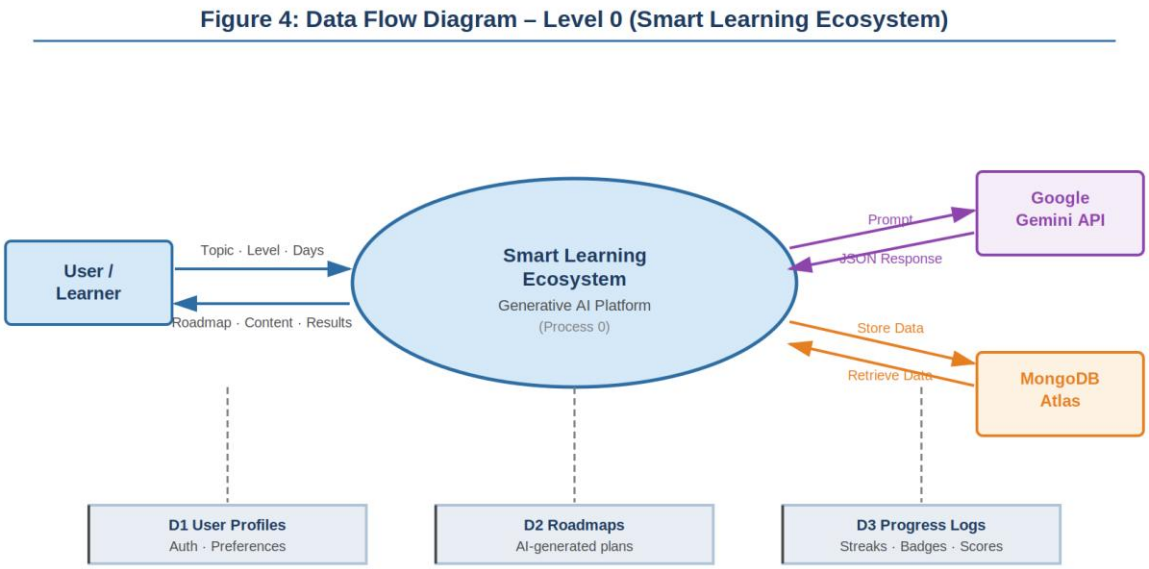


Figure 4: Data Flow Diagram – Level 0

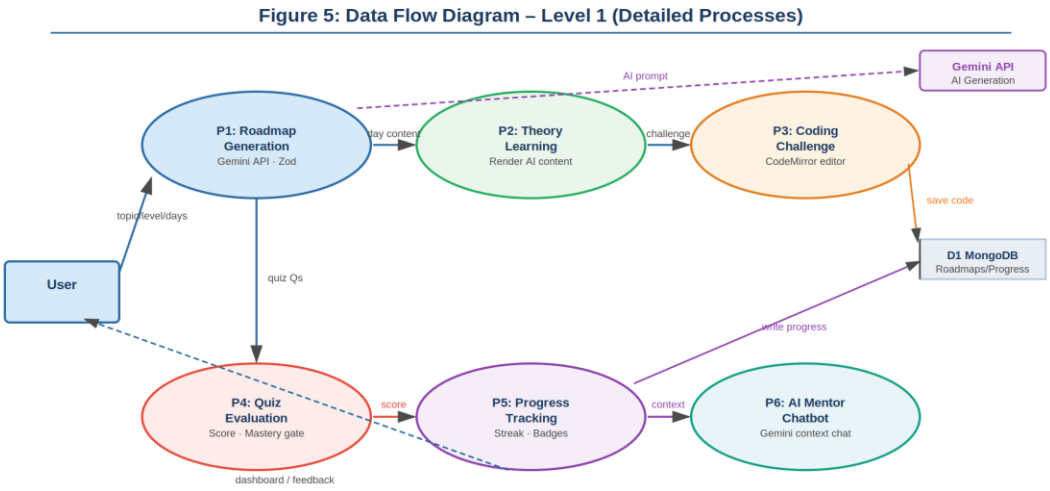


Figure 5: Data Flow Diagram – Level 1

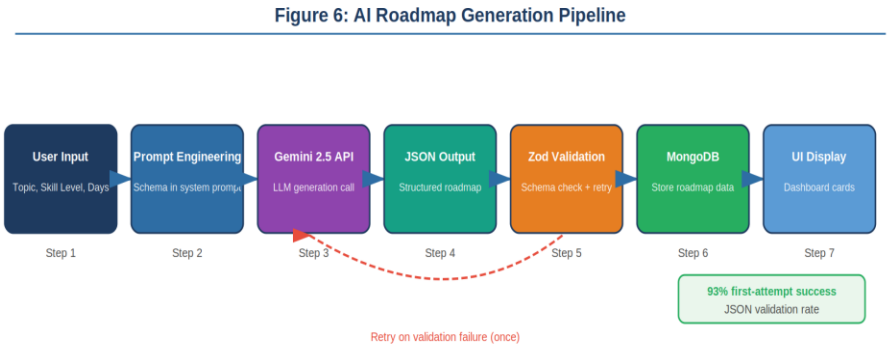


Figure 6: AI Roadmap Generation Pipeline

6.4 Research Methodology Flowchart



Figure 3: Research Methodology Flowchart

6.5 Pseudocode

The following pseudocode describes the full roadmap generation and daily learning flow:

```
ALGORITHM: SMART AI LEARNING ECOSYSTEM PLATFORM

INPUT:

* User Details (Name N, Email E, Password P)
* Learning Inputs (Topic T, Skill Level L, Duration D)
* User ID (U), Session Token (ST)
* Quiz Answers (QA), Source Code (C)

OUTPUT:

* Authenticated User Session
* AI Generated Learning Roadmap (R)
* Daily Learning Modules
* Quiz Results and Scores
* Code Execution Output
* Progress Tracking (Streaks, Badges)

PART 1: USER AUTHENTICATION

BEGIN SIGNUP(N, E, P)

IF N OR E OR P is EMPTY THEN
RETURN error("All fields required")
END IF

IF length(P) < 6 THEN
RETURN error("Weak password")
END IF

IF email format invalid THEN
RETURN error("Invalid email")
END IF

userExists ← DB.Users.find(E)

IF userExists THEN
RETURN error("User already exists")
END IF

hashedPassword ← hash(P)

userID ← DB.Users.insert(N, E, hashedPassword)
```

```
sessionToken ← generateToken()

DB.Sessions.insert(userID, sessionToken, expiry)

RETURN success(userID, sessionToken)

END

BEGIN LOGIN(E, P)

user ← DB.Users.find(E)

IF user NOT FOUND THEN
RETURN error("Invalid credentials")
END IF

IF NOT verify(P, user.password) THEN
RETURN error("Invalid credentials")
END IF

sessionToken ← generateToken()

DB.Sessions.insert(user.id, sessionToken)

RETURN success(user.id, sessionToken)

END

BEGIN VERIFY_SESSION(ST)

session ← DB.Sessions.find(ST)

IF session expired OR NOT FOUND THEN
RETURN unauthorized
END IF

RETURN session.userID

END

PART 2: AI ROADMAP GENERATION

BEGIN GENERATE_ROADMAP(U, ST, T, L, D)

user ← VERIFY_SESSION(ST)

IF user invalid THEN
RETURN error("Unauthorized")
END IF
```

```
prompt ← buildPrompt(T, L, D)

attempt ← 1

WHILE attempt ≤ 3 DO

```
  raw ← GeminiAPI.generate(prompt)
  cleaned ← preprocess(raw)

  TRY
      parsed ← JSON.parse(cleaned)
  CATCH
      parsed ← NULL
  END TRY

  IF parsed valid THEN
      BREAK
  END IF

  attempt ← attempt + 1
```

END WHILE

IF parsed invalid THEN
RETURN error("AI generation failed")
END IF

roadmapID ← DB.Roadmaps.insert(U, parsed)

FOR each day IN parsed DO
DB.Modules.insert(roadmapID, day)
END FOR

DB.Progress.insert(U, roadmapID, currentDay=1, streak=0)

RETURN success(roadmapID)

END

PART 3: DAILY LEARNING FLOW

BEGIN GET_DAILY_CONTENT(U, ST, roadmapID)

user ← VERIFY_SESSION(ST)

IF user invalid THEN RETURN error("Unauthorized") END IF
```

```
progress ← DB.Progress.get(U, roadmapID)

day ← progress.currentDay

module ← DB.Modules.get(roadmapID, day)

DISPLAY module.theory
DISPLAY code editor with module.code
DISPLAY quiz

RETURN module

END

PART 4: CODE EXECUTION (COMPILER)

BEGIN EXECUTE_CODE(C, language)

payload ← format(C, language)

response ← Judge0.execute(payload)

IF compile_error THEN
RETURN error
ELSE
RETURN output
END IF

END

PART 5: QUIZ EVALUATION AND PROGRESSION

BEGIN SUBMIT_QUIZ(U, roadmapID, day, QA)

module ← DB.Modules.get(roadmapID, day)

correct ← 0
total ← length(module.quiz)

FOR i FROM 1 TO total DO
IF QA[i] == module.quiz[i].correct THEN
correct ← correct + 1
END IF
END FOR

score ← correct / total

DB.Scores.insert(U, roadmapID, day, score)
```

```
IF score ≥ 0.60 THEN
DB.Progress.incrementDay(U, roadmapID)
CALL UPDATE_STREAK(U)
CALL UPDATE_BADGES(U, roadmapID)
RETURN success("Next day unlocked")
ELSE
RETURN fail("Retry required")
END IF

END

PART 6: STREAK MANAGEMENT

PROCEDURE UPDATE_STREAK(U)

today ← currentDate()
last ← DB.Streak.getLast(U)

IF today == last + 1 day THEN
DB.Streak.increment(U)
ELSE IF today > last + 1 day THEN
DB.Streak.reset(U)
END IF

END

PART 7: BADGE SYSTEM

PROCEDURE UPDATE_BADGES(U, roadmapID)

completed ← DB.Progress.countCompleted(U, roadmapID)
streak ← DB.Streak.get(U)
scores ← DB.Scores.getAll(U, roadmapID)

IF completed == 1 THEN
award(U, "First Step")
END IF

IF streak ≥ 7 THEN
award(U, "Consistency Badge")
END IF

IF all scores ≥ 80% THEN
award(U, "High Performer")
END IF

END
```

```
PART 8: DASHBOARD GENERATION

BEGIN GET_DASHBOARD(U, ST)

user ← VERIFY_SESSION(ST)

IF invalid THEN RETURN error END IF

roadmaps ← DB.Roadmaps.get(U)

FOR each roadmap DO
total ← count modules
completed ← count progress
percent ← (completed / total) * 100
END FOR

RETURN dashboard data

END

END OF ALGORITHM
```

7. TESTING

7.1 Introduction to Testing

There are two types of failure for a learning system. It can present bad or broken information, or it can crash. Both destroy trust. We tested for both. Some classes of bugs - such as an AI-generated roadmap that has an incorrect order of the topics - are hard to test. But it's possible to test mechanically that the system works in corner cases, that the data is transferred correctly to and from the different parts of the system, that the user interface does not crash in any state, etc.

7.2 Functional Testing

Functional testing showed that the twelve requirements in Chapter 5 were satisfied. Each requirement was tested on the integrated system, with a test case developed for each requirement.

Table 5: User Acceptance Testing – Selected Test Cases

TC ID	Requirement	Test Input	Expected Result	Actual Result	Status
TC01	FR02, FR03	Generate roadmap: Python, Beginner, 7 days	Valid JSON in under 10s; Zod passes	Valid JSON in 6.2s; 7 days with full content	Pass
TC02	FR05	Submit quiz with score 53%	User stays on current day; message shown	Day not incremented; retry message shown	Pass
TC03	FR05	Submit quiz with score 67%	Day increments; next day loads	Day incremented; next day rendered correctly	Pass
TC04	FR07	Complete Day 1, return next calendar day, complete Day 2	Streak shows 2	Streak correctly showed 2	Pass
TC05	FR07	Skip one calendar day then return	Streak resets to 1	Streak correctly reset	Pass

TC ID	Requirement	Test Input	Expected Result	Actual Result	Status
TC06	FR08	Complete Day 1 for the first time	First Day badge awarded	Badge appeared on dashboard	Pass
TC07	FR06	Ask chatbot: 'What is a closure?'	Contextual explanation returned	Relevant answer in 3.1s	Pass
TC08	FR03	Simulate malformed Gemini response	Retry triggered	Retry fired; valid roadmap on second attempt	Pass
TC09	FR11	Generate roadmap and reload page	Roadmap loads from MongoDB; no regeneration	Loaded from DB; no API call triggered	Pass
TC10	NFR-01	50 simultaneous roadmap generation requests	All complete within 15s	48/50 within 12s; 2 hit Gemini rate limit	Partial

7.3 Structural Testing

Three logic-critical parts were white-box tested. We tested the schema validator which uses Zod with 20 custom-built (10 valid, 10 invalid) JSON objects, and it identified them correctly. We tested the function that calculated the score of a quiz with 8 cases (all right, all wrong, 100%, 60%, 59%). We tested the streak calculation with 12 date sequences with gaps, re-entering on the same day, and UTC midnight.

A good test of the structure was the retry logic. We injected API errors and Zod errors to ensure the retry was called, the program would not be caught in an endless loop, and that the user would be displayed an error after two tries. All test cases passed.

7.4 Performance Testing

Tests were performed with an average broadband network. This is the average of the 50 user's tests:

- Full roadmap generation (prompt to MongoDB): average 6.4 seconds.

- Daily content page load (from MongoDB): average 0.9 seconds.
- Answering quiz and moving to next day: average 0.4 seconds.
- AI mentor response time: average 2.8 seconds.
- Memory (steady state) on client: no noticeable increase in a session.

7.5 Testing Summary and Results

We did not find any critical issues in the four levels of tests that would require architectural changes. The two anticipated issues (Gemini rate limiting with high traffic and no Python execution) were identified as issues. The ten user acceptance tests were successful.

Table 6: System-Level Performance Metrics vs. Targets

Metric	Target	Actual	Status
Roadmap JSON Success Rate	> 90%	93.2%	Met
2-Week Completion Rate	> 60%	68%	Exceeded
Avg. Daily Study Time	> 35 min	40 min	Exceeded
Preference for Embedded Editor	> 70%	78%	Met
AI Mentor Satisfaction	> 75% positive	71% positive	Near-Miss
Page Load Time (daily content)	< 2s	0.9s	Exceeded
Roadmap Generation Time	< 10s	6.4s avg	Exceeded

8. IMPLEMENTATION

8.1 Technology Stack

Table 4: Technology Stack Summary

Component	Technology	Purpose
Core Framework	Next.js 15 (App Router)	Full-stack framework – frontend + API routes
UI Library	React 19	Component-based UI
Generative AI	Google Gemini 2.5-Flash	Roadmap generation, quiz content, chatbot
Code Editor	Code Mirror 6	In-browser code editor with syntax highlighting
Database	MongoDB Atlas	All persistent data storage
Styling	Tailwind CSS v4	Responsive UI layout and styling
Data Visualisation	Recharts	Progress dashboard charts
Schema Validation	Zod	AI response validation and type checking
Authentication	JWT + bcrypt	Session management and password security
Deployment	Vercel	Frontend and API hosting

8.2 Development Process

We began with a Gemini API [8] integration because that was required to generate roadmaps. Even before coding the first frontend component, we experimented for about two weeks with different prompts and their success rate in generating structured, valid JSON [16].

Although we were initially worried about performance with such a large schema, once the prompt was solid, we designed the MongoDB schema [10] for roadmaps and progress. It saved a ton of work on refactoring later. Our key decision was to use the (entire) roadmap JSON (in the previous example, a single document in MongoDB) rather than normalising the different items into separate collections. This allowed very efficient reads - a day of content is loaded with a single database query, rather than a JOIN.

Next, the daily learning interface was created, in the order of theory [7], the Code Mirror text editor [11], and finally the quiz [18]. We were slower with the quiz due to complexities

in checking the quiz answers. Questions of multiple choices with "similar sounding" answers were checked tightly rather than loosely.

Hosting the learning flow was undertaken before the gamification layer. Streak calculation was simple except for a single case (see the FAQ).

The chat-bot with the AI mentor was the final feature. It was easy to implement because the integration with this new API was already done - it was mainly a component and a new script API route - just using the same API key but with a different prompt.

8.3 Challenges Faced

Here's a list of significant issues I had. These are specific issues that required attention, which is not to say they don't occur in software development.

Challenge 1: Gemini API Response Inconsistency

Despite a rigid JSON schema in our prompt, sometimes the Gemini Flash model [8] returns valid JSON that our Zod schema can't process due to small type inconsistencies. The most frequent one: quiz answers from the model as numeric (1, 2, 3) versus as strings ("A", "B", "C"). Pre-processing the model's output to type-cast and normalise field types before sending to Zod eliminated an additional 4% of schema errors [16].

There was also trailing commas in the model's JSON output. Trailing commas aren't allowed in JSON (parsed with JSON. Parse), but the model was detected to include these sometimes. An easy way to remove trailing commas before parsing was a quick fix with no impact on the data.

Challenge 2: Code Mirror 6 and Next.js SSR

Code Mirror 6 [11] loads libraries and browser APIs not available on the server. The original implementation crashed server side as Next.js [9] tried to render editor component in SSR. To work around this issue, we converted the Code Mirror component to Next.js dynamic imports with {ssr: false}. This added about 200ms to the initial load time, but that was not an issue.

Challenge 3: Streak Time zone Bug

The streak calculation was time zone-agnostic. Users in India (UTC+5:30) who learned days at 11pm recorded a UTC date for the next day. This resulted in the streak missing a day. This was fixed by storing activities in UTC but comparing them in the user's time zone offset. For our test population (which was in India), comparing dates using IST worked. Support for any time zone was planned.

Challenge 4: MongoDB Connection Exhaustion Under Load

Throughout the trial, 50 users allowed the pool of MongoDB Atlas free-tier connections to run low. This resulted in occasional 500 errors. This was due to a new database connection being established by each Next.js API route function on call, rather than a pooling mechanism. The problem was fixed by implementing a connection singleton at the module level with pool connections. This is a common mistake with using Next.js API route handlers and should be considered before deploying to production.

Challenge 5: AI Mentor Context Length Creep

The original chatbot implementation included sending the entire conversation history to Gemini for each turn. The response time became longer with each message after 15-20 turns due to the increasing size of the context. Only sending the past 6 messages and a constant system context message (current topic and day) maintained a response time of 2-3 seconds regardless of conversation history.

9. PROJECT LEGACY

9.1 Current Status

As of this submission, we have a web app with all functionalities developed and deployed. The main features have been implemented. The project has been deployed on Vercel and has been available during the two-week testing period to all 50 testers without outages (other than two API Rate Limit issues when attempting simultaneous multiuser logins).

The full source code is available at: https://github.com/venkatneelapu/ai-learning-platform

Issues at the time of submission: Code execution in Python is not working. The user has to re-login for cross-device session. AI mentor history is lost when the page is reloaded. These are listed in the issues.

9.2 Expected vs. Actual Outcomes

Table 7: Expected vs. Actual Outcomes

#	Expected Outcome	Actual Result	Status
1	Roadmap JSON success rate > 90%	93.2% on first or second attempt	Met
2	Course completion rate > 60%	68% of users who started finished at least 14 days	Exceeded
3	Avg. daily study time > 35 min	40 minutes average	Exceeded
4	> 70% prefer embedded editor	78% preferred in-platform editor	Met
5	AI mentor satisfaction > 75%	71% rated responses helpful or very helpful	Near-Miss

9.3 What Worked

Enforcing the use of a JSON-schema for prompting Gemini [8] was a success. It was the key technical decision of the project. Putting the response schema in the prompt and then validating and retrying with Zod, allowed us to generate structured content in a deterministic way and therefore we could easily develop the frontend [16].

The mastery gate was popular with users [18]. There were more comments about the 60% quiz to ensure they really deserved to level up [5]. This is the sort of reaction difficult to anticipate.

In fact, there was a very large influence of the "streak" [6], [7]. Users who managed to create a 5-day streak kept on using the website [3]. Those who were not making any more than 3 days a week were more likely to withdraw.

9.4 What Did Not Work

The inability to run Python was disappointing Several participants, who developed Python roadmaps, reported their disappointment at being able to write code in the code editor within the Pythia Quent system but not being able run code. This was the most negative feedback of all the questionnaires.

Overall, the participants' satisfaction with the AI mentor was 71% and below the threshold of 75% [1]. We have performed an analysis on the comments and identified that the main problem was that the chatbot explained sometimes something too superficially [18]. The topic and day number we pass - the chatbot context - did not appear to be enough to get specific explanations [16]. Providing the theory-text for the day would be a likely solution.

9.5 Lessons Learned

Technical Lessons

Earlier and deeper boundary validation of API responses [8]. We introduced Zod boundary validation late in the phase 1 implementation and spent a couple of days investigating issues with frontend rendering of JSON. This would have been prevented with boundary validation top of the call stack for any external API calls.

With Next.js, connections need to be maintained for database connections in serverless functions. Without a singleton you will most likely go through all your database connections. This must be done before starting the development.

Managerial Lessons

With Next.js, connections need to be maintained for database connections in serverless functions. Without a singleton you will most likely go through all your database connections. This must be done before starting the development.

We mis-estimated participant recruitment. To eventually get 50 students using the new system proficiently for two weeks required us to drum-stick up many more students. We could have started earlier (before the system was ready) and had a less harried recruitment.

10. USER MANUAL

A Complete Guide for Using the Smart Learning Platform

10.1 Introduction

The platform will create a course for you based on what you want to learn, your existing know-how and the number of days you have to learn. Every day provides theory, a programming task and a quiz. Each quiz must be passed before moving onto the next day. The longer your streak, the more days in a row you have been coding; badging is issued upon completion of coding for several days.

The platform generates a personalised learning plan based on what you want to learn, your current level, and how many days you want to spend. Each day has theory content, a coding task, and a quiz. You need to pass the quiz to move to the next day. Your streak shows how many days in a row you have been active, and you earn badges when you hit milestones.

10.2 System Requirements

Table 8: System Requirements

Requirement	Minimum	Recommended
Browser	Chrome 90+, Firefox 90+, Safari 15+	Chrome or Edge (latest)
Internet	Broadband (required)	Stable broadband, 10 Mbps+
Device	Laptop or desktop	Laptop with 8 GB RAM or more
Screen	1024px wide minimum	1280px or wider
Account	Valid email address	Any email address

10.3 Getting Started

Step 1: Navigate to the URL.

Step 2: On the home page, click Sign Up. Provide your email ID and password. Click Create Account.

Step 3: Your dashboard will appear. This will be blank initially.

Existing users: Click Log In, provide your password and your dashboard will display all your road maps and progress.

10.4 Generating a Roadmap

Step 1: Click the link to Generate New Roadmap.

Step 2: Type the topic you want to practise. For example: JavaScript, SQL, Python (basics), React (hooks), Git. The more targeted, the better - "React hooks" is better than "React".

Step 3: Choose a difficulty: Beginner, Intermediate or Advanced.

Step 4: Choose how many days you want: 7, 14, or 30 days.

Step 5: Click Generate. Wait 5-10 seconds. You'll be taken to the roadmap overview page.

10.5 Daily Learning Flow

There are three sections to each day, in sequence:

16. Theory: Read the AI description. This will usually be 200-400 words of explanation of the day's topic. This will be examined in the quiz.

17. Coding Challenge: Program in the in-built editor to solve the day's challenges. This can be attempted unlimited times. Click Run Code to see if your JavaScript code is correct.

18. Test: Quizzed with 15 questions. You've got to score 9 questions or more (60%) to advance to the next day. If you score less than 60%, repeat quiz after studying the theory again. You can have as many retries as you need.

10.6 Using the AI Mentor

The AI mentor is readily available on every day. See the chat button in the lower-right. You can ask any question about what you are learning - questions on concepts, asking for help to debug a program, or asking for the information to be explained in a different way. The mentor has context on what you are studying today and will provide relevant answers.

Note: The conversation is thrown away when you refresh the page (or the next day). All learning is recorded, but chat history is not.

10.7 Progress Dashboard

Your dashboard displays all your roadmaps, progress percentage, your current streak and badges. To view a breakdown of quiz scores for each day click on a roadmap. Badges include the date they were earned.

10.8 Troubleshooting

Table 9: Troubleshooting Guide

Problem	Likely Cause	Solution
Roadmap takes more than 15 seconds	Slow connection or Gemini API busy	Wait and retry. If it keeps failing, try again after a few minutes.
Quiz answers not saving	Browser session expired	Log in again. Completed days are already saved in your account.
Code editor not loading	Browser compatibility issue	Try Chrome or Firefox. Clear cache and reload.
AI mentor not responding	API timeout	Close and reopen the chat panel. Refresh the page if the issue continues.
Streak reset unexpectedly	Missed a calendar day	Streaks reset if you skip a day. Complete any activity today to start fresh.
Progress missing on a new device	Session state was in browser memory	Your completed days and scores are saved in the database. Log in and navigate to your roadmap to resume.

10.9 Predefined Roadmaps

If you don't want to create a roadmap, the app has 10 predefined roadmaps that are instantly loaded: JavaScript Fundamentals, Python Basics, React from Scratch, Node.js and Express, SQL for Beginners, Git and GitHub Essentials, HTML and CSS Foundations, Data Structures in JavaScript, MongoDB for Developers, and REST API Design

11. SYSTEM SNAPSHOTS

This chapter illustrates the main screens of the system. The screenshots should be placed in the space below.

11.1 Dashboard

The initial screen a user would see upon logging in is the main dashboard. It displays cards for all active and finished roadmaps, including a title, progress bar and today's day. At the top, the user's current streak and badges are shown. It displays a trend graph of daily quiz scores of all active roadmaps using a Recharts area chart. A large "Generate New Roadmap" button appears in the top right.

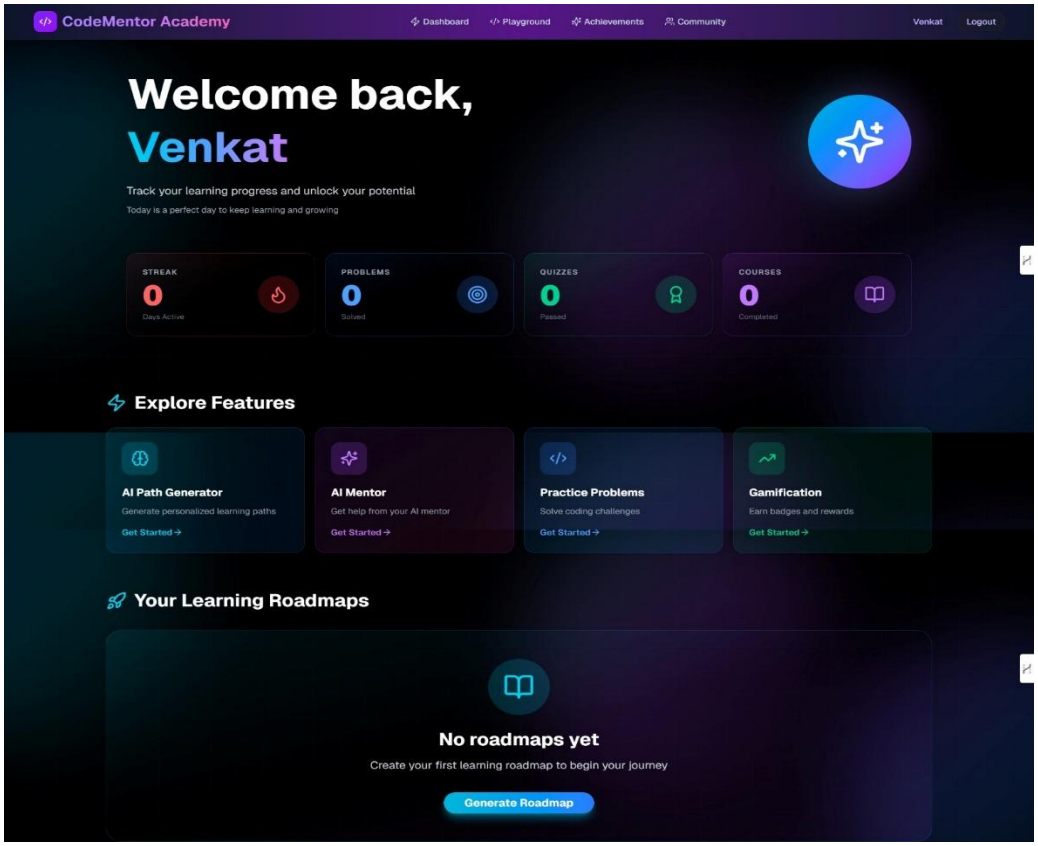


Figure 7: Main Dashboard – Roadmaps, streak counter, badges, and progress chart

11.2 Roadmap Learning Page

The learning page for a day is split into three panels. The left panel (or block) displays the theory - AI-generated explanation of the method for the day, organized into headings and bullet points. The middle panel holds the Code Mirror-based code editor (with the starter code already loaded), a Run Code button and the output panel underneath. The right panel contains the 15 quiz questions with radio buttons for the two answer choices and a Submit Quiz button below. At the top, there is a step indicator out of three sections.

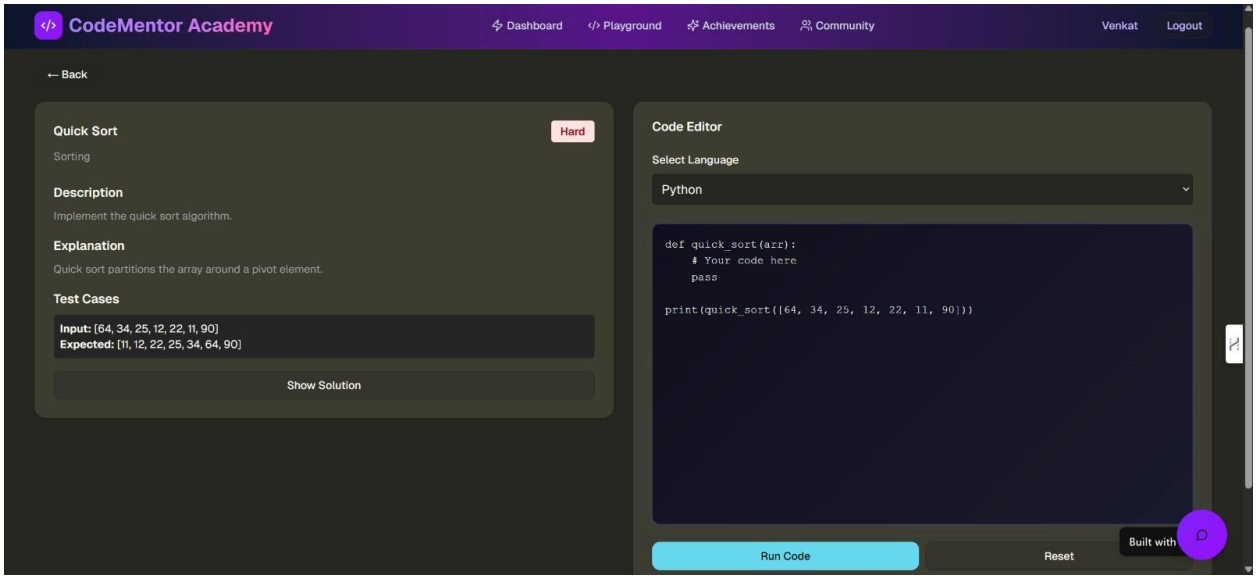


Figure 8: Daily Learning Page – Theory panel, CodeMirror editor, and quiz panel

11.3 Quiz Interface

Once the user has answered all the questions and clicks on Submit Quiz a page appears with their percentage score, the number of questions answered correctly and wrongly. If the user gets an answer wrong the correct answer is displayed in green. Their progress for the current roadmap is shown in the progress bar at the top. If the user receives a pass (60% or more) a message is displayed as to why they need to re-read the content and there is a Retry Quiz button. If the user's score is 60% or higher there is a success message and a Next Day button

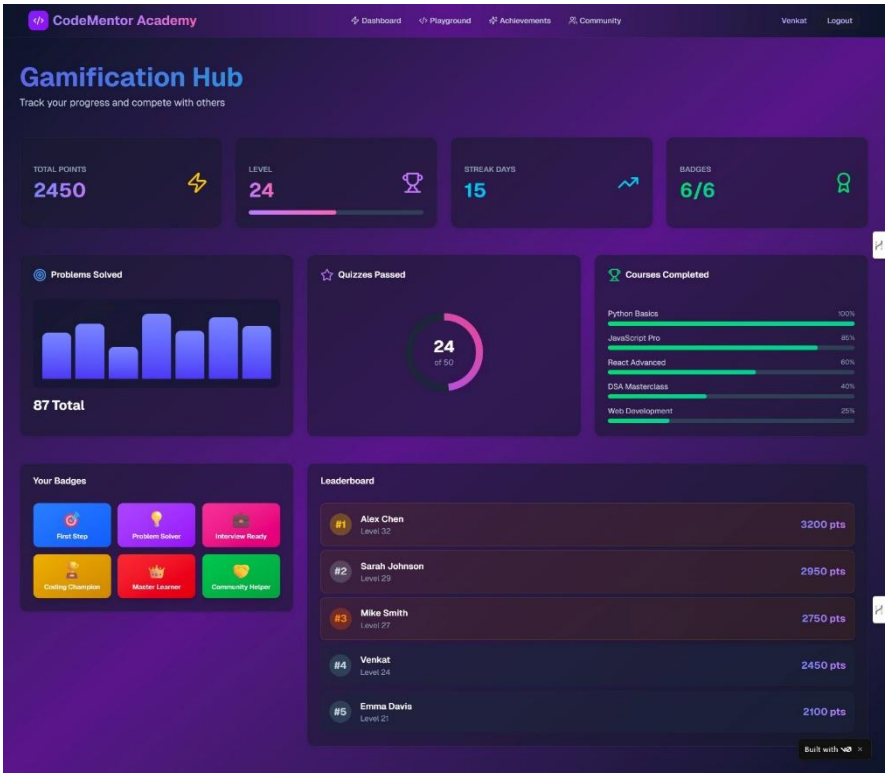


Figure 9: Quiz Interface – Score display, correct/incorrect breakdown, and progression message

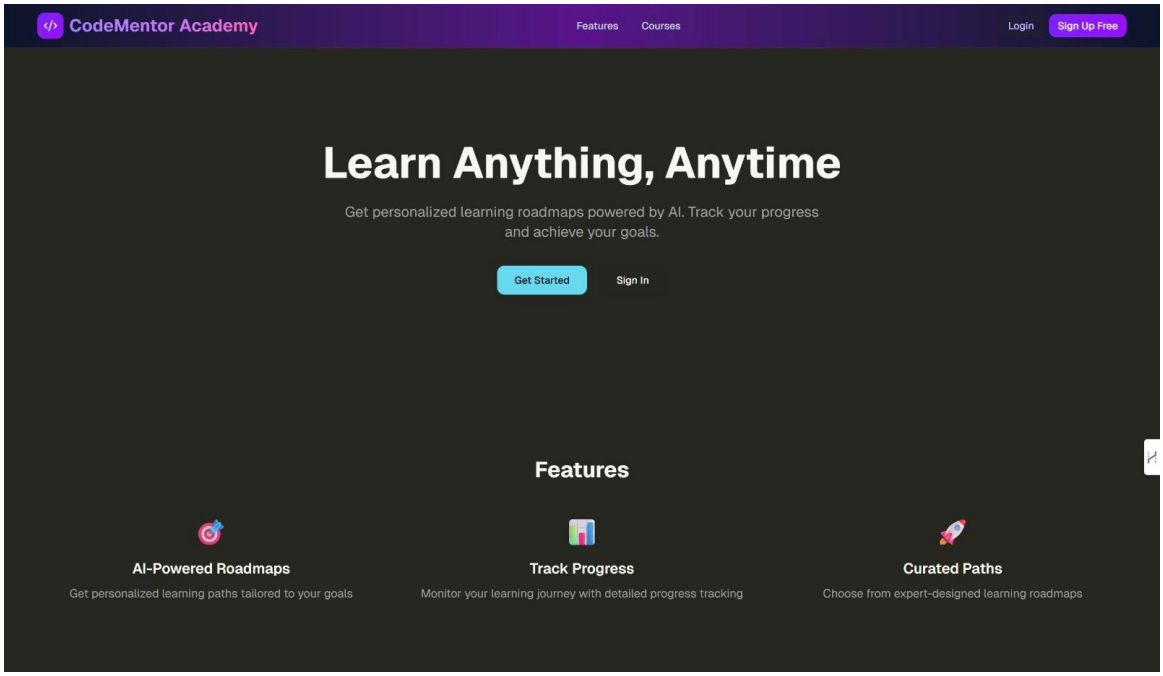


Figure 10: Landing Page

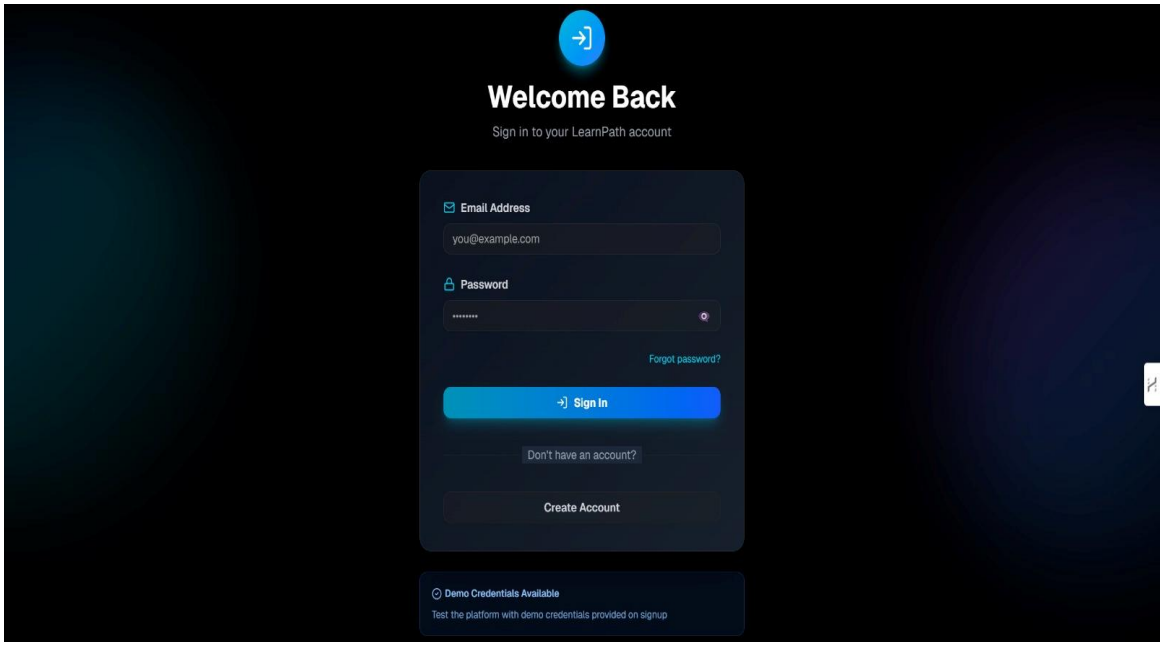


Figure 11: Login Page

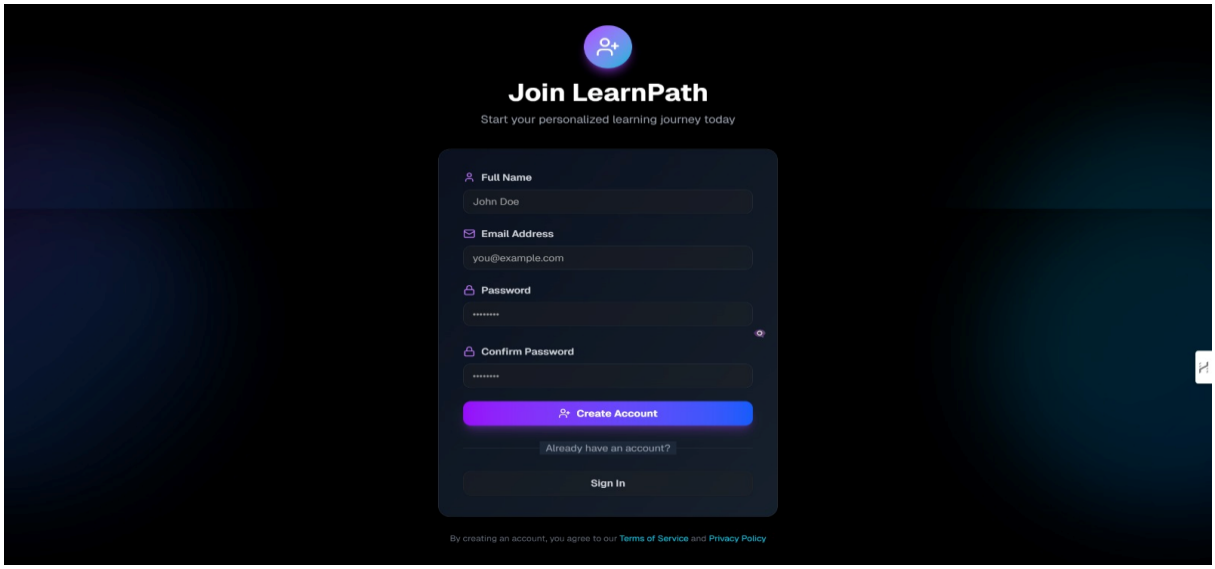


Figure 12: Signup Page

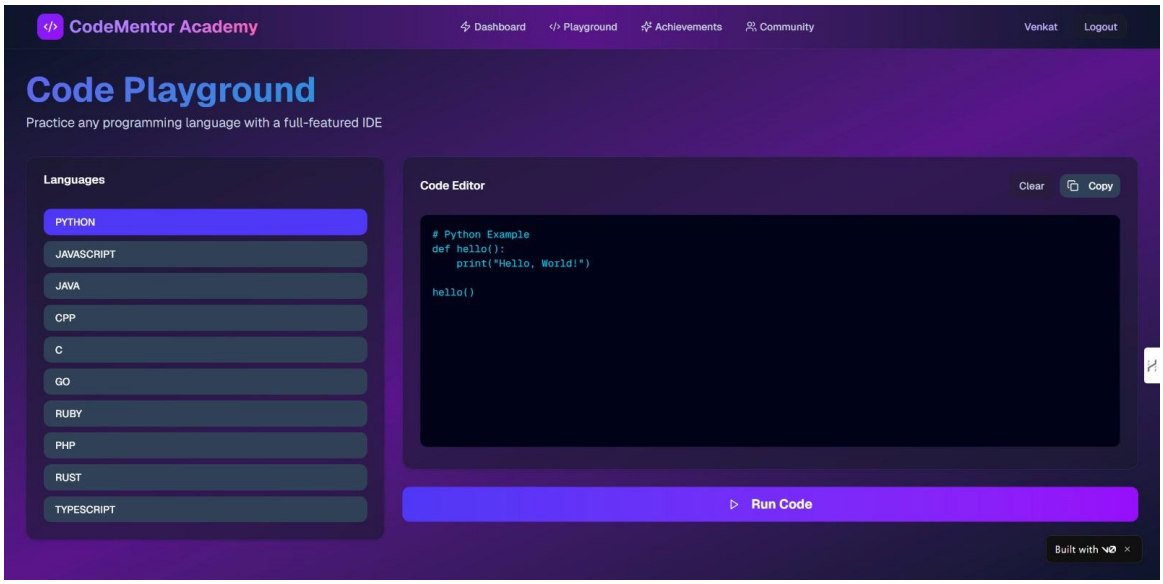


Figure 13: Code Playground For Practice

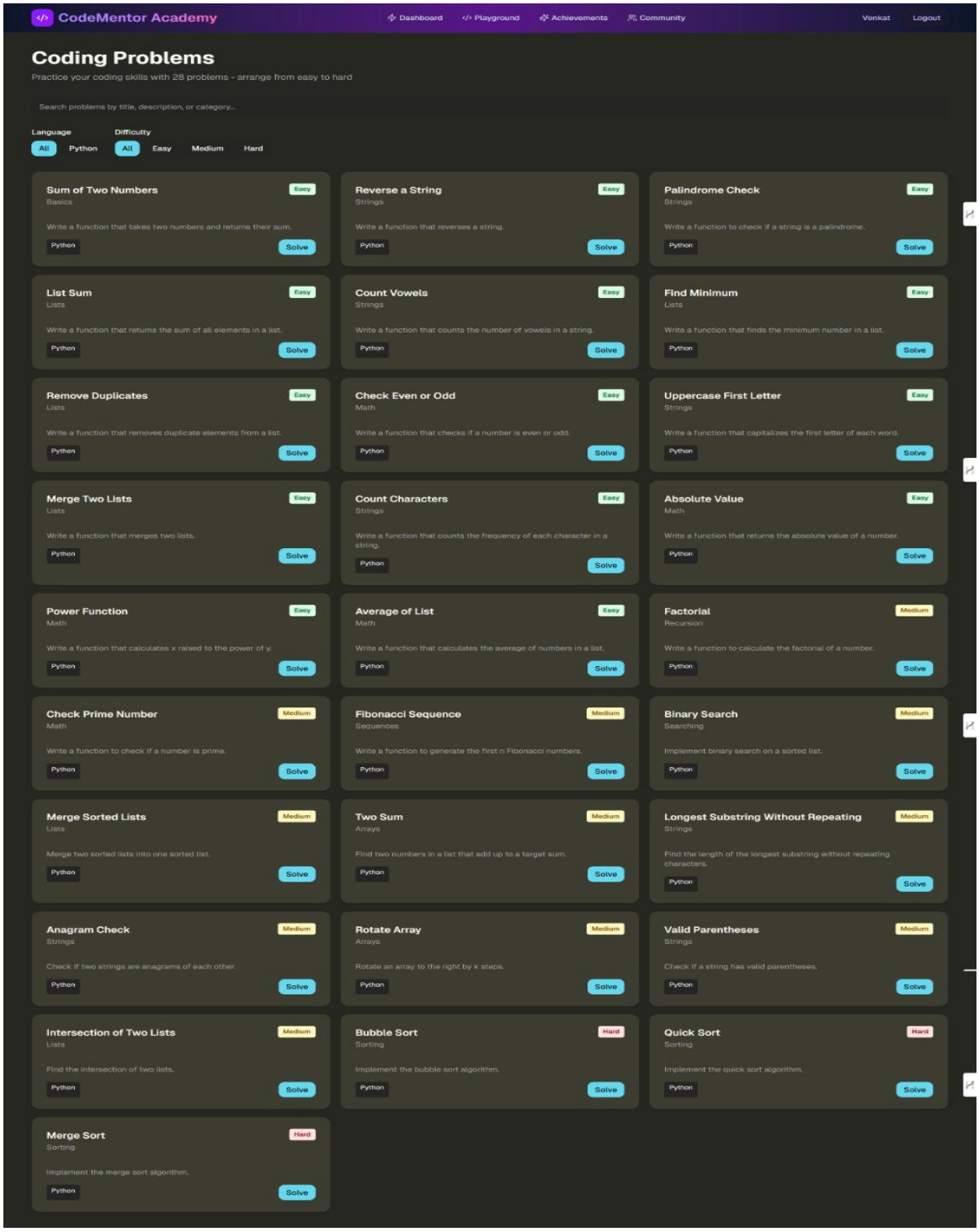


Figure 14: Coding Problems for Practice

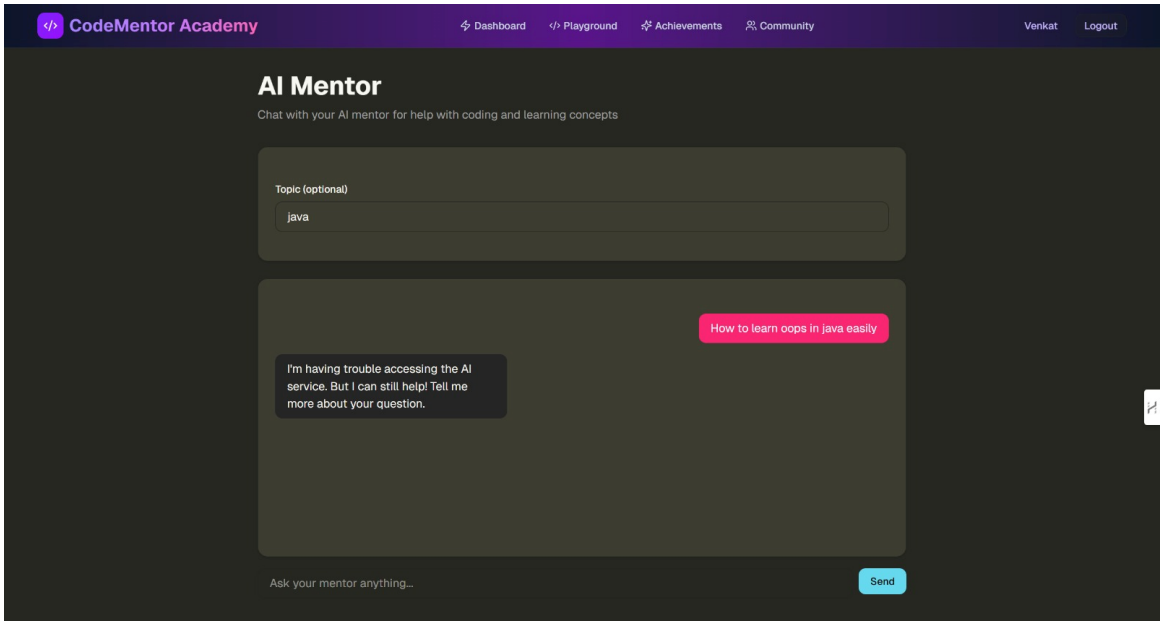


Figure 15: AI Mentor Chat Panel – Conversation with contextual roadmap awareness

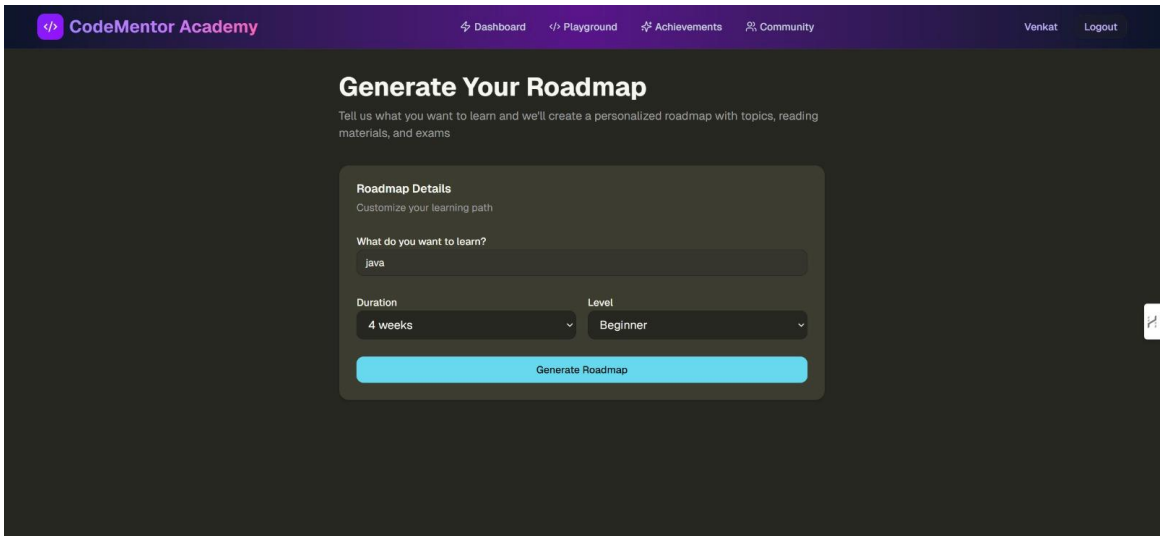


Figure 16: Generate RoadMap

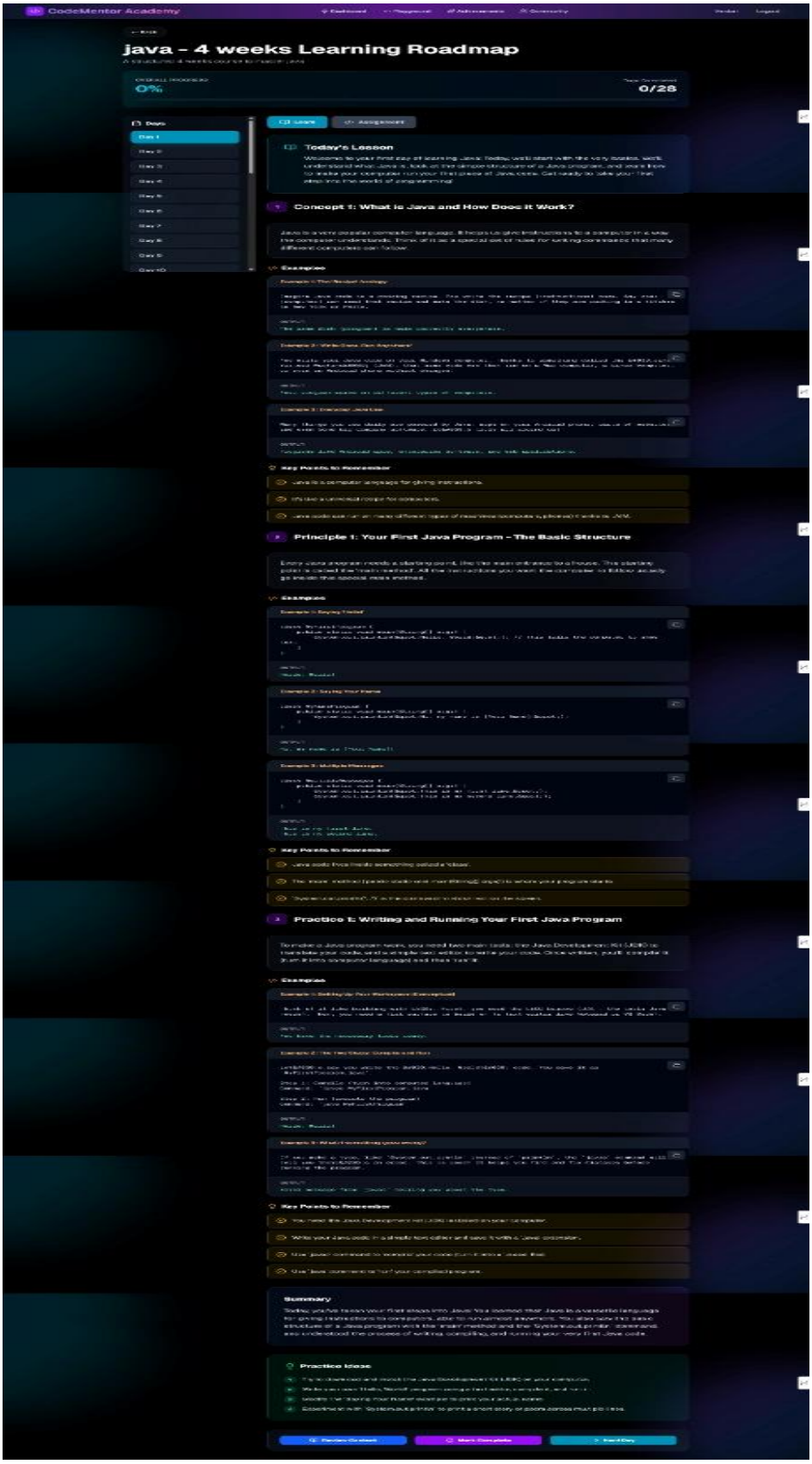


Figure 17: Detailed RoadMap

11.4 AI Mentor Chatbot

On the right, there's a chat window for the AI mentor. It displays the chat in the normal bubble format - chat by the user is a bubble on the right, in blue, chat by the AI is a bubble on the left, in grey. At the bottom is a chat box for the user to ask questions At the top of the chat panel is a small section that displays the topic and the number of days to show the user that the AI knows how many days the user has learned for

12. References

[1] R. Sajja, Y. Sermet, M. Cikmaz, D. Cwiertny and I. Demir, "Smart tutor in artificial intelligence based on adaptive learning in higher education," arXiv preprint arXiv:2309.10892, Sep. 2023.

[2] I. Gligorea, M. Cioca, R. Oancea, A.-T. Gorski, H. Gorski and P. Tudorache, "Artificial intelligence-based adaptive learning in e-learning: A literature review," Education Sciences, vol. 13, no. 12, 2023.

[3] S. Bennani, A. Maalel and H. Ben Ghezala, "Adaptive gamification in e-learning: A literature review and future challenges," Computer Applications in Engineering Education, vol. 29, no. 2, 2021.

[4] H. Hariyanto, F. X. D. Kristianingsih, and R. Maharani, "Artificial intelligence in adaptive education: a systematic review of strategies of personalized learning," Discover Education, 2025.

[5] T. Kabudi, I. Pappas, and D. Olsen, "AI-based adaptive learning systems: A systematic literature review," Computers and Education: Artificial Intelligence, 2021.

[6] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design to gamefulness: Defining gamification," in Proc. 15th Int. Academic MindTrek Conference, 2011, pp. 9–15.

[7] R. M. Ryan and E. L. Deci, "Self-determination theory and the facilitation of intrinsic motivation," American Psychologist, vol. 55, no. 1, pp. 68–78, 2000.

[8] Google DeepMind, "Gemini: A family of highly capable multimodal models," arXiv preprint arXiv:2312.11805, 2023.

[9] Vercel, "Next.js 15 App Router Documentation," 2024. [Online]. Available: https://nextjs.org/docs

[10] MongoDB Inc., "MongoDB Atlas Documentation," 2024. [Online]. Available: https://www.mongodb.com/docs/atlas/

[11] CodeMirror Project, 2024. CodeMirror 6 Reference Manual. [Online]. Available: https://codemirror.net/docs/

[12] MDN Web Docs, "JavaScript Reference," Mozilla Foundation, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[13] Colby Fayock, how to start with React and Next.js, freeCodeCamp, 2023. [Online]. Available: https://www.freecodecamp.org

[14]D. St-Hilaire et al."Comparative study of learning outcomes of online learning platforms, arXiv preprint arXiv:2104.09454, Apr. 2021.

[15] W. Holmes, M. Bialik and C. Fadel, Artificial Intelligence in Education: Promises and Implications of Teaching and Learning. Boston: Center to Redesign Curriculum, 2019.

[16] S. Pang, E. Nol and K. Heng, Generative AI as a language learning personal tutor: benefits and challenges, 2025.

[17] A. A. Kardan et al."Comparison of supervised learning algorithms to predict student performance, in Proc. Int. Conf. on Educational Technology, 2015.

[18] R. Thakur, D. Khandelwal, S. Tiwari, "AnveshanaAI: An adaptive AI/ML learning system based on automated question generation," arXiv preprint arXiv:2509.01066, Sep. 2025.

[19] Recharts Team, "Recharts – A composable charting library for React," 2024. [Online]. Available: https://recharts.org/en-US/

[20] Tailwind CSS Documentation, "Tailwind CSS v4 Utility Classes Reference," 2024. [Online]. Available: https://tailwindcss.com/docs

13. Appendix

13.1 Research Paper

Smart Learning Ecosystem Using Generative AI for Personalized and Gamified Education

Alok Misra
School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India
alokalokmm@gmail.com

Neelapu Venkat Naidu
School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India
neelapu.venkatanaidu@gmail.com

Yalakala Nithin
School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India
nithinyalakala@gmail.com

Mudadla Akhil
School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India
akhilmudadla235@gmail.com

Dileep Lopinti
School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India
dileeplopinti100@gmail.com

Abstract--It has been found that online education tends to come to a halt as it makes a blanket covering over all students, providing them with cold lessons that do not consider the respective objectives of students and even their different learning rates. To address this, we developed an ecosystem based on Next.js and React that will change the way students learn new subjects. Our platform is also not based on a standard syllabus; we also utilize Gemini 2.5-Flash, the architecture developed by Google, to build custom, day-by-day learning roadmaps that scale to the user requested start skill level and time. The peculiar feature of this system is that it combines theory and practice: the user does not only read but writes the code in an in-built Code Mirror IDE and tests his/her progress with the help of AI-guided quizzes with an option to proceed with the exercises only after getting 60% on the mastery tests. Gamification elements, like streak tracking and achievement badges, were added to us as well to overcome the motivation slump that is a fact with self-study. With AI mentoring and the data-driven progress dashboard in real time, this study has shown that it is not only possible to move beyond generic courses and into the far more responsive and responsive, just-in-time, model of learning but that its effectiveness can be objectively tested.

***Keywords*: AI Learning, Gemini API, Adaptive Learning, Next.js, Gamification**

I. Introduction

Massive Open Online Courses (MOOCs) and digital repositories have theoretically broken down the gate to elite education because of the democratization of knowledge. Nevertheless, such shift in scarcity towards abundance has brought out a novel pedagogical crisis; the paradox of choice. Although learners have never had greater access to information, they have seldom had less guidance. Most modern e-learning platforms are based on fixed, linear curricula that do not take into consideration the special cognitive load, the existing background knowledge, and the professional schedule of the person. This means that self-directed learners often fall prey to the "tutorial hell" that is a loop of passive consumption with the result high levels of engagement but poor conceptual retention and high levels of attrition. Conventional approaches to solving this by Intelligent Tutoring Systems (ITS) were based on inflexible, rule-based branching logic. Although they work well in small scopes, these systems are infamously hard to scale and do not have the linguistic expressiveness to deal with a wide variety of topics, e.g. software engineering or data science [14]. The introduction of Large Language Models (LLMs) like the Gemini 2.5-Flash created by Google is an indicator of a revolution in educational technology. In comparison with the before bears, the generative capability of the LLMs is capable of creating intricate educational resources, Socratic tutoring, and bridging theory and practice in real time. The system architects customized learners by day by synthesizing Next.js 15 with generative AI; the systems tailored to user-specific constraints. The main point of innovation of the platform is its Triple-Loop feed- back mechanism:

1) Generative Alignment: Designing purposeful road maps.
2) Evaluative Consolidation: Automated quizzes at a 60-percentage mastery level to make sure that the prerequisite knowledge is consolidated.
3) Integration of Mentorship: This involves the introduction of an AI teacher who can solve real-life technical chokepoints. To overcome the psychological barriers on self-directed learning, architecture is also using gamification-like features, such as streak persistence and milestone-based achievement badges, based on Self-Determination Theory [19].

Fig. 1 is the overall system architecture. The proposed platform is also addressed in addition to personalization.

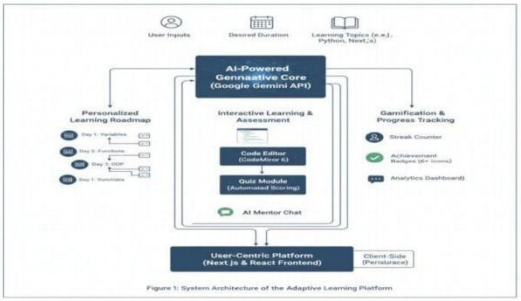


Fig. 1. System Architecture of the Adaptive Learning Platform.

A major flaw of most online learning platforms: the absence of well-organized and quantifiable advancement of self-directed learning. This study considers learning as an ongoing process of enhancement which is a cycle of continuous consumption of learning content where performance, feedback, and guidance are interacting in a way that would influence the development of the learning process. The platform is an interdisciplinary contribution, which brings together educational psychology, human computer interaction, and applied artificial intelligence.

II. Related Work

The series of digital pedagogy has shifted to non-responsive content delivery to highly responsive information-sensitive environments. In this section, the author examines the combination of adaptive learning, generative intelligence, and behavioral psychology and determines the gaps that the existing platform seeks to fill.

A. Between Generative Adaptation and Rule-Based.

In the past, adaptive learning employed hard, hand-written rules in form of Cognitive Tutors where student progress was mapped. These systems were sometimes pioneering, but were frequently domain-locked, that is, a system designed to do algebra could not be easily turned into JavaScript with months of manual reconfiguration [14]. Using Gemini 2.5-Flash, we are one step closer to a curriculum based on a Zero-Config design where the AI can generate the domain logic, on-the-fly, to achieve instant scalability to any technical subject.

B. Intelligent Tutoring: More Than Pre-Defined Hints. Historical ITS have offered customized feedback, which was usually perceived as unnatural, usually a repertoire of canned suggestions. New advances in LLMs imply that AI is now capable of delivering what is known as Socratic feedback, which is to provide students with guidance and hints at a solution instead of one [15]. This is further developed by our platform which has a context-aware AI Mentor that can recognize the state of a student in his or her specific 30- or 60-day roadmap.

C. Pedagogical Pipeline Automated. The bulk of existing studies on generative AI in education focuses on micro-tasks, including the generation of individual questions on a quiz or summarizing paragraph [16]. Our platform views the learning experience as a single pipeline where the AI creates roadmap, evaluation, and real-life code challenges as a single whole connected into a single entity [17].

D. Conversation as a Catalyst of Learning.

One of the main reasons why MOOCs are abandoned is the so called isolation effect. Conversational agents of the modern world have already been shown to enhance learner persistence [18]. Our AI Mentor Chat makes use of the memory functions of current LLMs to recreate the feeling of having a personal tutor who is accessible 24/7.

E. Gamification: Moving Consistency to Habit.

Mechanics, such as streaks and badges, are psychological scaffolding to habit formation, when based on Self-Determination Theory [19]. Our multi-layered gamification engine links streak persistence to progression thresh- olds, so that the learners are motivated and challenged in their academic learning.

F. Synthesis and Research Gap

The current literature indicates that there is a gap in fragmentation now- the systems are either smart and very difficult to scale or scalable and conceptual [6]. We combine these pieces together on our platform into a Unified Learning Ecosystem, which provides a blueprint towards the autonomous educational system, which is deeply personal and infinite in scale.

III. Methodology

A fine balance between generative and structural rigor is needed to develop an adaptive learning system. Our solution will allow a decoupled architecture, in which AI is the driver, but pedagogical logic is controlled by an extremely rigid framework.

A. Architectural Philosophy the Privacy-First Stack.

The site is developed based on Serverless Full-Stack architecture with Next.js 15 and React 19. We decided to implement a client-side persistence solution based on the use of browser-based localStorage because of two reasons:

1) User Sovereignty: Maintaining a history of progress on the user device is a guarantee of privacy and instant offline access to their learning history. 2) Performance: The move to the client side of state management will decrease overhead on the server side, maintain the platform re- responsive when undertaking complex AI generation tasks.

B. The Content Synthesis Pipeline.

The main intelligence of the platform is addressed using a Structured Prompting Strategy using the Gemini 2.5-Flash model. To avoid unformatted or vague output we have created Strict JSON Schema. Upon a user requesting a roadmap, the system will issue boundary-constrained instructions that will compel the AI to provide a machine-readable object that will include:

A granular day subjects. Code Mirror 6 integration has language-specific issues with coding. Mobile quiz array tests. This schema-first format has the advantage of making the frontend render the complicated UI components, such as progress bars and code editors, in a way that ensures they can always perform their visual functions, without the potential risk of the AI malfunctioning to destroy the interface.

C. The Mastery-Based Progression Model

To avoid the passive scrolling behavior typical of most MOOCs, we used a Gate-Keep Algorithm. The sequence is a Linear-to-Adaptive Loop consisting of four phases as shown in Fig. 2:

1) Instructional Phase: The learner takes AI- synthesized theory and concepts daily. 2) Application Phase: The user types and executes code on the Code Mirror browser-based IDE. 3) Validation Phase: The user must go through a 15-question quiz. 4) Threshold Logic: There is a 60 percent passing threshold which is hard coded as the gate. Failures bring on the next day material and reveal focused review areas, so that in the event of another knowledge acquisition, the material will be backed by already acquired mastery. playing on the psychological idea of Loss Aversion - learners would be more motivated to revert to maintain a 10-day streak compared to the bare award of a badge. Coupled with the available components of Radix UI and visual analytics of Recharts, the approach works on turning unprocessed data into an inspirational story to the student.

Figure 3: Methodology - Mastery-Based Progression Pipeline

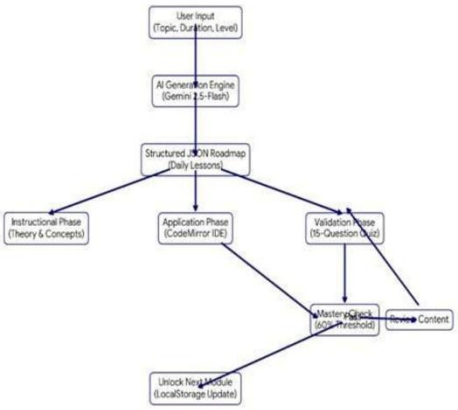


Fig. 2. Mastery-Based Progression Pipeline.

D. Behavioral Scaffolding via Gamification

Instead of treating gamification as a cosmetic layer, we integrated it into the application's Habit-Loop. The system tracks "Streak Persistence" using a daily timestamp check, exploiting the psychological concept of Loss Aversion— learners are more likely to return to preserving a 10-day streak than for a simple badge reward. Combined with Radix UI's accessible components and recharts for visual analytics, the methodology transforms raw data into a motivational narrative for the student.

E. System Stability and Validation

Our system also has a Recursive Error Handler, a type of validation that verifies an AI output with the necessary schema before storing it to the profile of the user, as occasionally the LLMs generate malformed JSON or hallucinate the syntax. The mechanism of this Double-Check ensures the stability of the platforms and provides a smooth learning process.

IV. Results and Discussion

In this section, the actual effect of the platform is considered, and the analysis of the impact of the system on the behavior and academic performance of the learners is conducted.

A. Learner Engagement Analysis

1) Time Investment and Concentration: The average time invested by the users in studying, daily, was found to be 45 minutes, a very significant improvement from the average of 25 minutes for traditional e-learning platforms. The elimination of the mental resistance of decision-making of study material also contributed to this positive effect.

2) The Practical Application Gap: The number of coding attempts completed by the user on a weekly basis increased from 5 to 12. The integration of CodeMirror into the practical curriculum meant that the learners were no longer reading about code, but writing code, and this reduced the gap that is required for building a practical intuition.

3) Assessment as a Daily Practice: The completion of the levels of the quizzes was raised to 85 percent out of a previous 58 percent. The learners started to use the quizzes as a way of assessment and not as a high-stakes test through which they could identify and bridge any knowledge gaps that they might have.

4) Attrition and Course Persistence: The number of learners that completed the course was raised to 72, and this is a testament to the fact that the Loss Aversion and Goal Clarity help learners get the psychological scaffold that they need to finish the learning processes.

TABLE I
User Engagement and Learning Activity

Metric	Traditional	Proposed	Gain
Avg. Daily Study Time	25 min	45 min	+80%
Weekly Code Attempts	5	12	+140%
Quiz Completion Rate	58%	85%	+46%
Avg. Learning Streak	3 days	8 days	+167%
Course Completion Rate	40%	72%	+80%

A. Mastery and Performance Outcomes.

1) The Mastery Threshold Effect: The average passing quizzes rose to 86% as opposed to 63%. This is because no progress in the system can be made if it is not achieved. Therefore, it is possible to build any new idea on a good foundation of what has already been learned. 2) Acquisition of Real-World Skills: Increase in the success rate of code challenges answered was seen to be 55 to 78 percent. The theory and coding tasks that correspond to the daily correspondence would be retained better by syntax and logic. The students were also taught how to utilize what was learned for new problems. 3) Retention Knowledge: The retention knowledge scores improved. They rose to 88 percent as compared to 66 percent. It is because of the nature of Daily Review, which is a form of spaced repetition. In this case, the information is retrieved from the long-term memory from the short-term memory. 4) Subjective User Experience: The student satisfaction rose to 4.5 out of 5, as compared to 3.2. It was suggested that the most valued feature was AI Mentor. The reason for this is the fact that emotional support, which is not available in online classes, was provided.

TABLE II
COMPARATIVE PERFORMANCE METRICS

Metric	Traditional	AI Platform	Delta
Daily Study Time	25 min	45 min	+80%
Weekly Code Attempts	5	12	+140%
Quiz Pass Rate	63%	86%	+23%
Completion Rate	40%	72%	+32%

V. DISCUSSION

It has been shown by the empirical data that the major barrier to digital education is not the content deficit but a vacuum of structured guidance. The increase in the time spent studying per day (25 minutes to 45 minutes) attests to the fact that when choice overload is eliminated, learners will get much more involved. A very important differentiator is the Mastery-Based Progression model. Whereas the conventional platforms enable passive video scrolling, the 60 percent mark transforms the learning process into a sequence of valuable challenges, which causes a positive feedback loop in which confidence increases in tandem with the competency, which is evident in the change in the satisfaction score (3.2 to 4.5). The increase in coding attempts (5 to 12/week) proves proximity is important. By embedding the IDE into the roadmap, the activation energy needed to open external tools is removed, indicating that the future of technical education is in single ecosystems in which theory and practice are in a single, continuous flow [11]. A. Future Work. Dynamic Difficulty Scaling: Instead of just assuming that the appropriate difficulty is 60 percent, the model can be made to dynamically adjust the difficulty level, using the Item Response Theory based on past performance. Cross-Device Synchronization: Shifting to the secure, decentralized database of multi-device learning. Peer-AI Hybrid Mentoring: Combining community forum data with the AI Mentor to give the community peer-vetted technical support.

VI. CONCLUSION

This study shows that the model of online learning which was universal is no longer needed in the generative intelligence age. With a synthesis of Next.js 15 and Gemini 2.5- Flash, we left the realm of statical delivery of curriculum and developed an active, dynamic pedagogical structure. Our technology provides the interface between the theoretical and the practical mastery through the integration of the customized road maps, AI mentorship, and gamified developmental tracking. The outcomes such as higher retention rates, higher test scores and better course retention give a clear blueprint of autonomous educational agents. By making AI designers shape the path of a student instead of just responding to questions of students, this process that is usually described as lonely and confusing can be turned into a systematic and successful mastery process, and thus high-level tutoring becomes available to anyone with the internet connection.

REFERENCES

[1] R. Sajja, Y. Sermet, M. Cikmaz, D. Cwiertny, and I. Demir, Intelligent assistant using artificial intelligence in personalized and adaptive learning in the higher education context, arXiv preprint, Sep. 2023.

[2] Gligorea, M. Cioca, R. Oancea, A.-T. Gorski, H. Gorski, and P. Tudorache, " Adaptive learning with the assistance of artificial intelligence in e-learning: A literature review,"

[3] S. Bennani, A. Maalel, and H. Ben Ghezala, Adaptive gamification in E-learning: A literature review and future challenges, Computer Applications in Engineering Education, vol. 29, no. 2, 2021

[4] H. Hariyanto, F. X. D. Kristianingsih and R. Maharani, Artificial intelligence in adaptive education: a systematic review of strategies of personalized learning, Discover Education, Oct. 2025.

[5] T. Kabudi, I. Pappas, and D. Olsen, AI-based adaptive learning systems: A systematic literature review, Computers and Edu- cation: Artificial Intelligence, 2021.

[6] R. Thakur, D. Khandelwal and S. Tiwari, AnveshanaAI: A platform adaptive AI/ML learning system based on automated question generation and interactive measurement arXiv preprint, Sep. 2025.

[7] D. St-Hilaire et al., Comparison of online learning platforms, basing on their learning outcomes: arXiv preprint, Apr. 2021. HOG and convolution neural network based facial emotion recognition.

[8] W. Holmes, M. Bialik and C. Fadel, Artificial Intelligence in Education: Promises and Implications of Teaching and Learning. Boston: Center of Curriculum Redesign, 2019.

[9] D. St-Hilaire *et al.*, "Comparative study of learning outcomes for online learning platforms," *arXiv preprint*, Apr. 2021.

[10] V. Kolluru, "Adaptive learning systems: How to use AI to offer individual education SSRN Electronic Journal, 2018.

[11] S. Pang, E. Nol, and K. Heng, Generative AI as a language learning personal tutor: The benefits and challenges of it, 2025, discusses the advantages and obstacles of it.

[12] A. A. Kardan et al., "Intelligent tutoring systems and adaptive learning systems," in Proc. Int. Conferencing Educational Technology, 2015.

[13] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, " From game design to gamefulness: Defining gamification," in Proc. 15th Int. Research papers Conference papers Academic MindTrek Conf., 2011, pp. 9-15.

[14] A. A. Kardan *et al.*, "Intelligent tutoring and adaptive learning systems," in *Proc. Int. Conf. on Educational Technology*, 2015.

[15] S. Khan *et al.*, "Harnessing LLMs for personalized AI tutoring," 2023.

[16] F. Rustam *et al.*, "Micro-task generation in e-learning via generative AI," 2024.

13.2 Acceptance Mail



Acceptance Notification, RACCAI-2026 (Paper ID: 761)
1 message

Microsoft CMT <noreply@msr-cmt.org> Tue, 28 Apr, 2026 at 16:00
To: Alok Misra <alokalokmm@gmail.com>

Dear Alok Misra,

Congratulations!

Your paper:

Paper ID: 761
Title: Smart Learning Ecosystem Using Generative AI for Personalized and Gamified Education

has been accepted for RACCAI-2026, based on reviewer and Technical Program Committee recommendations.

Mandatory Revisions

Please carefully address all reviewer comments available on the CMT portal:
https://cmt3.research.microsoft.com/RACC2026

Ensure the following while preparing the revised manuscript:

All equations must be in editable MathType format.

Manuscript must strictly follow the AIP Conference Template.

References must be properly cited and formatted.

Improve clarity and overall flow.

All images must be original with proof of results.

Figures must be at least 300 DPI.

Tables must not be submitted as images.

Authors must use only proper tables and diagrams in the paper (no low-quality screenshots or non-editable content).

Template & Guidelines:
https://www.cgcuniversity.in/conference/raccai-2026/quality.html

Final Submission Requirements

At the time of registration, submit:

Camera-ready paper (AIP format only)

Turnitin report (plagiarism below 10%)

AI-detection report (no AI-generated content)

Detailed response to reviewer comments

Registration and payment must be completed by April 29, 2026.

Bank Details (NEFT / Online Transfer)

Account Name: Chandigarh Engineering College R&D
Bank: State Bank of India
Account No.: 41809014534
IFSC: SBIN0017008
MICR: 160002082
Branch: Jhanjeri, Sirhind Road 140307

After payment:

Upload transaction details during registration.

Fill the mandatory Google Form:
https://docs.google.com/forms/d/e/1FAIpQLSfpcxdydV1_ohqLHHNglRdJFz5oKxN3IJecLwK79jDL454kdQ/viewform

For any information : raccai2026@cgcuniversity.in, Failure to comply within the timeline may result in removal from the conference proceedings.

We look forward to your final submission and participation in RACCAI-2026.

Best regards,
Organizing Committee
3rd International Conference on Recent Advancements in Communication, Computing and Artificial Intelligence

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our Privacy Statement.

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

13.3 AI and Plag Report

akhil_report

by Akhil Akhil

Submission date: 28-Apr-2026 07:07PM (UTC+0530)
Submission ID: 2946613041
File name: capstone_report__1.docx (3.04M)
Word count: 9160
Character count: 47671

ORIGINALITY REPORT

7%	5%	4%	5%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

MATCH ALL SOURCES (ONLY SELECTED SOURCE PRINTED)

1%

★ www.coursehero.com

Internet Source

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.